Transformer Encoder Frankenstein: Biblioteca, CLI y Notas de Diseño Basadas en Investigación

Erick F. Merino M.
Este trabajo no está afiliado
erickfmm@gmail.com

Febrero 2026

Resumen

Transformer Encoder Frankenstein presenta un kit de herramientas unificado basado en configuración para la experimentación sistemática con arquitecturas de codificador modernas, abarcando diecisiete variantes de mezclador de secuencias y veinte familias de optimizadores. Las contribuciones de investigación son triples: (i) un contrato de configuración basado en esquema estricto que permite la experimentación reproducible en diversos mecanismos de atención, incluyendo atención softmax estándar, atención sigmoide, redes retentivas, modelos de espacio de estado selectivos, transformadores de profundidad continua, atención aumentada con memoria, patrones de atención dispersa y mecanismos con compuerta; (ii) un marco integral de enrutamiento de optimizadores que admite métodos de reducción de varianza (MARS, Adan, AdEMAMix), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programador y precondicionadores de segundo orden (Shampoo, SOAP, Sophia); y (iii) flujos de trabajo de extremo a extremo que abarcan el despliegue cuantificado mediante empaquetado de pesos ternarios y entrenamiento de incrustaciones de oraciones inspirado en SBERT. El kit de herramientas implementa una interfaz de configuración basada en web que proporciona renderizado de formularios impulsado por esquemas con documentación en línea y validación en tiempo real. Este documento de referencia técnica incluye diagramas arquitectónicos, visualizaciones de flujo de ejecución, tablas de decisión y apéndices integrales que sintetizan la literatura sobre arquitecturas de transformadores, mecanismos de atención dispersa, variantes de atención con compuerta y algoritmos de optimización. El sistema permite iteración rápida mientras mantiene condiciones experimentales reproducibles a través de su filosofía de diseño basada primero en esquemas.

Índice

1. Introducción **3**
1.1. Motivación y Planteamiento del Problema . . . 3
1.2. Contribuciones . . . 4
1.3. Guía de Lectura . . . 4
1.4. Constructor de Configuración Basado en Web . . . 5

2. Trabajo Relacionado **6**
2.1. Arquitecturas de Mezclador de Secuencias . . . 6
2.1.1. Líneas Base de Atención Densa . . . 6
2.1.2. Arquitecturas Recurrentes y Retentivas . . . 7
2.1.3. Mecanismos de Atención Dispersa . . . 8
2.1.4. Mecanismos de Memoria con Compuerta . . . 8
2.2. Algoritmos de Optimización Avanzados . . . 9

3. Arquitectura Centrada en Configuración **10**
3.1. Contrato de Esquema y Validación . . . 10
3.2. Validación de Esquema . . . 10

4. Taxonomía de Arquitectura e Implementación **10**
4.1. Familias de Atención y Mezclador de Secuencias . . . 10
4.2. Atención Estándar . . . 11
4.3. Atención Sigmoide . . . 11
4.4. Formulación Retentiva . . . 12
4.5. SSM Selectivo (Mamba) . . . 12
4.6. Actualizaciones Continuas Estilo ODE . . . 12
4.7. Memoria en Tiempo de Prueba (Titans) . . . 12
4.8. Extensiones Dispersas y con Compuerta en la Base de Código Actual . . . 12

5. Familias de Optimizadores y Dinámica de Entrenamiento **12**
5.1. Organización Taxonómica de Optimizadores Soportados . . . 13
5.2. Formulación Adaptativa Central . . . 13

6. Cuantización y Despliegue **13**
6.1. Cuantización Ternaria . . . 14
6.2. Cuantización de Activaciones . . . 14
6.3. Estimaciones de Tamaño . . . 14

7. Tareas Posteriores SBERT **14**

8. Tablas Resumen **15**
8.1. Resumen de Atención y Mezclador de Secuencias . . . 15
8.2. Resumen de Optimizadores . . . 15

9. Discusión **16**
9.1. Compromisos de Diseño Impulsado por Esquemas . . . 16
9.2. Cobertura Arquitectónica y Brechas . . . 16
9.3. Fragmentación del Paisaje de Optimizadores . . . 17
9.4. Consideraciones de Despliegue y Producción . . . 17
9.5. Desafíos de Integración y Extensibilidad . . . 17

10. Conclusión **17**

Bibliografía **18**

1. Introducción

1.1. Motivación y Planteamiento del Problema

Las arquitecturas de transformadores han transformado fundamentalmente el panorama del modelado de secuencias en el procesamiento del lenguaje natural [35], visión por computadora y biología computacional. El éxito de modelos como BERT [9], GPT y sus variantes ha establecido al transformador como el estándar de facto para el aprendizaje de representaciones en aprendizaje profundo. Sin embargo, la rápida proliferación de innovaciones arquitectónicas presenta desafíos prácticos significativos para investigadores y profesionales.

Los años recientes han presenciado una explosión de mecanismos alternativos de atención y mezcla de secuencias, cada uno abordando limitaciones específicas de la atención softmax estándar: complejidad computacional cuadrática [7], requisitos de inferencia eficiente en memoria [33, 10], gestión de estado selectivo basada en contenido [2] y optimizaciones conscientes del hardware [27]. Simultáneamente, la literatura de optimización se ha diversificado más allá del AdamW clásico, introduciendo técnicas de reducción de varianza [39, 24, 45], variantes eficientes en memoria [30, 49], enfoques sin programador [8] y métodos de precondicionamiento de segundo orden [12, 36].

La consecuencia práctica es un ecosistema de investigación fragmentado donde la comparación experimental a través de arquitecturas y optimizadores requiere un esfuerzo de ingeniería significativo. Los investigadores deben implementar y depurar múltiples variantes desde cero, garantizar canalizaciones de entrenamiento consistentes y gestionar espacios de hiperparámetros complejos. Esta fragmentación obstaculiza la reproducibilidad, ralentiza el progreso científico y aumenta la barrera de entrada para nuevos investigadores.

Este trabajo aborda estos desafíos a través de un kit de herramientas de experimentación unificado basado en configuración que proporciona:

1. **Diseño Basado Primero en Esquemas**: Un contrato de configuración estricto y validado que impone reproducibilidad mientras admite diecisiete arquitecturas distintas de mezclador de secuencias y veinte familias de optimizadores.

2. **Entrenamiento Agnóstico a la Arquitectura**: Infraestructura de entrenamiento común que admite líneas base de atención densa (estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, bloques estilo ODE), patrones de atención dispersa (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).

3. **Marco de Enrutamiento de Optimizadores**: Grupos de hiperparámetros con prefijo que permiten control granular sobre incrustaciones, capas de normalización, bloques recurrentes, bloques de atención y otros subconjuntos de parámetros a través de optimizadores diversos.

4. **Flujos de Trabajo de Extremo a Extremo**: Despliegue integrado mediante cuantización (empaquetado de pesos ternarios, activaciones INT8) y capacidades de incrustación de oraciones inspirado en SBERT [28].

5. **Configuración Interactiva**: Interfaz basada en web que proporciona renderizado de formularios impulsado por esquemas, validación en tiempo real y generación de comandos CLI.

La superficie de comandos del proyecto es:

```
frankestein-transformer
```

con subcomandos: `train`, `deploy`, `quantize`, `infer`, `sbert-train`, `sbert-infer`, `web-server`.

El comando `web-server` lanza un constructor de configuración basado en Streamlit que proporciona:

- Campos de formulario impulsados por esquemas con títulos de parámetros y descripciones detalladas
- Información sobre herramientas y texto de ayuda en tiempo real para cada opción de configuración
- Vista previa y descarga en tiempo real de YAML
- Comandos CLI generados para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT

Esta interfaz interactiva sirve como alternativa a la edición manual de YAML, mejorando la usabilidad para usuarios que exploran opciones de configuración disponibles y comprenden su impacto en el comportamiento del modelo y la dinámica de entrenamiento.

1.2. Contribuciones

Este trabajo realiza las siguientes contribuciones principales:

1. **Esquema de Configuración Unificado**: Un contrato de esquema basado en YAML con validación estricta que admite diecisiete arquitecturas distintas de mezclador de secuencias a través de cuatro categorías (líneas base densas, bloques recurrentes/retentivos, patrones de atención dispersa y mecanismos con compuerta) mientras impone reproducibilidad a través de restricciones `additionalProperties: false`.
2. **Soporte de Arquitectura Integral**: Implementación de variantes modernas de transformador incluyendo atención softmax estándar [35], atención sigmoide [27], RetNet [33], Mamba [10], transformadores continuos estilo ODE [47], atención aumentada con memoria Titans [2], mecanismos de atención dispersa [7, 3, 46, 21, 44, 48, 38] y arquitecturas de atención con compuerta [41, 42, 43, 25, 17, 26].
3. **Marco de Enrutamiento de Optimizadores**: Sistema de hiperparámetros con prefijo que permite control por grupo de parámetros a través de veinte optimizadores, incluyendo métodos de reducción de varianza (MARS, Adan, AdEMAMix, Cautious AdamW), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programador (Schedule-Free AdamW), métodos conscientes de la curvatura (Sophia), precondicionadores de segundo orden (Shampoo, SOAP) y optimizadores orientados a la ortogonalidad (Muon, Turbo-Muon).
4. **Cuantización y Despliegue**: Canalización de despliegue integrada que admite empaquetado de pesos ternarios y cuantización de activaciones INT8 con estimaciones de tamaño siguiendo la aproximación de almacenamiento de 1,58 bits.
5. **Flujos de Trabajo de Incrustación de Oraciones**: Canalizaciones de entrenamiento e inferencia inspiradas en SBERT que admiten puntuación de similitud, recuperación, agrupamiento y exportación de incrustaciones persistentes.
6. **Interfaz de Configuración Interactiva**: Servidor web basado en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y generación de comandos CLI.

1.3. Guía de Lectura

Este documento está organizado como referencia técnica que aborda cuatro preocupaciones operacionales:

1. **Contrato de Configuración**: La sección 3.1 describe el esquema YAML que impone experimentos válidos y la sección 3.2 explica las reglas de validación.

2. **Selección de Arquitectura**: La sección **??** cubre opciones de normalización; la sección 4.1 proporciona comparación integral de familias de mezcladores de secuencias; los apéndices **??**, **??** y **??** sintetizan literatura de apoyo.

3. **Dinámica de Optimización**: La sección 5 detalla el enrutamiento de optimizadores y la dinámica de entrenamiento; el apéndice **??** proporciona análisis integral de familias de optimizadores.

4. **Despliegue e Inferencia**: Las secciones 6 y 7 describen el despliegue cuantizado y los flujos de trabajo de incrustación de oraciones.

1.4. Constructor de Configuración Basado en Web

Además de la edición directa de YAML, este proyecto proporciona una interfaz web basada en Streamlit (accedida a través del comando `web-server`) que mejora la accesibilidad y descubrimiento de configuración. La interfaz presenta campos de esquemas con:

- **Renderizado de formularios impulsado por esquemas** — Todos los campos se generan dinámicamente desde el esquema autoritativo, garantizando consistencia y validación.
- **Documentación de parámetros en línea** — Cada campo de formulario muestra un *título* del esquema como su etiqueta, con la *descripción* mostrada como ayuda al pasar el cursor.
- **Vista previa de configuración en tiempo real** — Los usuarios ven salida YAML en vivo mientras modifican campos de formulario, permitiendo retroalimentación de validación inmediata.
- **Generación de comandos CLI** — La interfaz genera comandos CLI completos para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT basados en la configuración actual.
- **Mejoras de accesibilidad** — Información sobre herramientas y formularios estructurados hacen que las opciones de configuración sean más fáciles de entender, especialmente para usuarios nuevos en el proyecto o explorando arquitecturas novedosas.

Este enfoque basado en web aborda barreras de usabilidad comunes en la experimentación impulsada por configuración:

- Reduce la necesidad de memorizar la estructura de YAML y los nombres de campos
- Previene errores tipográficos a través de validación de esquema
- Proporciona contexto educativo a través de documentación en línea
- Permite experimentación rápida con ajuste guiado de parámetros
- Sirve tanto como herramienta de configuración como recurso de aprendizaje

La implementación de la interfaz web utiliza widgets de formulario de Streamlit con:

- `st.checkbox()` para interruptores binarios con texto de ayuda
- `st.number_input()` para campos numéricos con tamaño de paso y formato
- `st.selectbox()` para opciones de enumeración con visualización de opciones
- `st.multiselect()` para selecciones de matriz desde opciones definidas

- `st.text_input()` para campos de texto
- `st.info()`, `st.caption()` para información complementaria

Los metadatos del esquema (campos de título y descripción) se extraen y renderizan sistemáticamente a través de todas las secciones de formulario, incluyendo:

- Parámetros de arquitectura del modelo (tamaño oculto, capas, cabezas de atención, etc.)
- Configuraciones de tiempo de ejecución de entrenamiento (tamaño de lote, acumulación, programador)
- Configuración de optimizador con hiperparámetros por grupo de parámetros
- Opciones de despliegue y cuantización
- Parámetros de entrenamiento e inferencia específicos de SBERT

2. Trabajo Relacionado

Este trabajo se sitúa en la intersección de tres áreas de investigación activas: arquitecturas de atención alternativas, mecanismos de atención dispersa y algoritmos de optimización avanzados. Esta sección sintetiza literatura relevante a través de estos dominios para contextualizar las contribuciones.

2.1. Arquitecturas de Mezclador de Secuencias

La trayectoria del modelado de secuencias en inteligencia artificial ha sido moldeada por una tensión continua entre expresividad computacional y eficiencia de recursos. El advenimiento del mecanismo de atención estándar de transformador transformó fundamentalmente el procesamiento del lenguaje natural, visión por computadora y biología computacional priorizando la contextualización global sobre la recurrencia secuencial. Sin embargo, a medida que la ambición de los modelos fundacionales escala hacia ventanas de contexto de millones de tokens, la complejidad computacional cuadrática $\mathcal{O}(n^2)$ de la autoatención tradicional ha surgido como un cuello de botella severo [35].

2.1.1. Líneas Base de Atención Densa

Atención Softmax Estándar. El mecanismo de autoatención multi-cabeza estándar estableció un cambio de paradigma eliminando completamente la recurrencia secuencial y convoluciones localizadas. Al permitir que cada token en una secuencia atienda directamente a cualquier otro token, el transformador estándar logró un éxito sin precedentes en capturar dependencias de largo alcance. La formulación calcula una suma ponderada de vectores de valor:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Para la decodificación autorregresiva, este mecanismo requiere almacenamiento en caché de Key-Value (KV) para evitar la recomputación $\mathcal{O}(n^2)$. Sin embargo, esta eficiencia teórica viene con el costo de una huella de memoria que crece linealmente requiriendo espacio $\mathcal{O}(n \cdot d)$, que puede consumir cientos de gigabytes de Memoria de Ancho de Banda Alto (HBM) para modelos grandes [35].

Atención Sigmoide. La atención sigmoide reemplaza la normalización softmax por filas con activación sigmoide elemento por elemento:

$$\text{SigmoidAttn}(Q, K, V) = \sigma\left(\frac{QK^\top}{\sqrt{d_k}} + b\right)V$$

Los análisis teóricos que aprovechan la perspectiva de mezcla de expertos revelan ventajas críticas en complejidad muestral. Para modelos que utilizan expertos ReLU, la compuerta softmax converge a $\mathcal{O}(n^{-0,24})$, mientras que la versión sigmoide logra una convergencia significativamente más rápida $\mathcal{O}(n^{-0,51})$. La naturaleza elemento por elemento evita la sobrecarga de sincronización entre hilos, permitiendo una aceleración del kernel de inferencia del 17 % mediante FlashSigmoid en GPUs NVIDIA H100 [27]. Sin embargo, el escalado empírico reveló inestabilidades de entrenamiento que requieren una capa de estabilización de "norma híbrida" para modelos a gran escala.

2.1.2. Arquitecturas Recurrentes y Retentivas

RetNet (Red Retentiva). RetNet fue diseñada explícitamente para resolver el "triángulo imposible" del modelado de secuencias: lograr entrenamiento paralelizable, inferencia de bajo costo $\mathcal{O}(1)$ y alto rendimiento simultáneamente. El mecanismo de retención introduce una matriz de decadencia exponencial causal fija D:

$$\text{Retention}(X) = (QK^\top \odot D)V$$

con forma recurrente:

$$S_n = \gamma S_{n-1} + k_n^\top v_n, \quad o_n = q_n S_n$$

Esta representación dual permite entrenamiento paralelo con complejidad $\mathcal{O}(n^2 \cdot d)$ e inferencia recurrente con tiempo $\mathcal{O}(1)$ por paso, eliminando completamente el caché KV [33].

Mamba (Modelo de Espacio de Estado Selectivo). Mamba aborda las limitaciones de los modelos de espacio de estado de Tiempo Invariante Lineal (LTI) introduciendo selectividad a los parámetros fundamentales. El sistema de tiempo continuo:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t)$$

se discretiza con parámetros dependientes de entrada:

$$\bar{A}_t = \exp(\Delta_t A), \quad \bar{B}_t = \Delta_t B_t, \quad \Delta_t = \text{softplus}(\text{Linear}(x_t))$$

El algoritmo de escaneo consciente del hardware permite complejidad lineal $\mathcal{O}(n \cdot d)$ con utilización eficiente de GPU, haciendo que la recurrencia lineal sea práctica en aceleradores [10].

Transformadores Continuos Estilo ODE. El transformador ODE trata la profundidad como integración numérica sobre un sistema dinámico continuo:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t)$$

con refinamiento Runge-Kutta que reduce el error de truncamiento y mejora la calidad de generación de secuencias. Esto proporciona una trayectoria intra-bloque más expresiva con uso compartido de pesos, aunque a mayor costo computacional [47].

Atención Aumentada con Memoria (Titans). Titans introduce memorización neuronal en tiempo de prueba: en lugar de almacenar solo activaciones, el modelo actualiza su memoria interna utilizando señales de aprendizaje impulsadas por sorpresa durante la inferencia. Esto aborda el contexto más allá de lo que los modelos recurrentes de estado fijo típicamente soportan, permitiendo contextos extremadamente largos y recuerdo asociativo. El costo del sistema es una mayor complejidad de implementación porque la inferencia ahora incluye lógica de adaptación de memoria [2].

2.1.3. Mecanismos de Atención Dispersa

La complejidad cuadrática de la atención estándar se vuelve prohibitiva para secuencias que exceden cientos de miles de tokens. Los mecanismos de atención dispersa abordan esto restringiendo el conjunto de interacciones de tokens, explotando la observación de que la mayoría de los pesos de atención aprendidos están cerca de cero.

Sparse Transformer. Introduce atención dispersa factorizada reduciendo la complejidad $\mathcal{O}(n^2)$ a $\mathcal{O}(n\sqrt{n})$. Utiliza dos patrones dispersos complementarios: atención con zancada y atención fija, cada una atendiendo a $\mathcal{O}(\sqrt{n})$ posiciones [7].

Longformer. Introduce atención de ventana deslizante con tokens globales opcionales, logrando escalado lineal $\mathcal{O}(n \cdot w)$ donde w es el tamaño de ventana fijo. Combina ventanas deslizantes, patrones dilatados y tokens globales para mantener conectividad de largo alcance con cómputo disperso [3].

BigBird. Combina ventanas locales, enlaces aleatorios y tokens globales para preservar una fuerte conectividad de contexto largo con cómputo disperso. Máscara dispersa aleatoria más rutas locales/globales mantiene propiedades teóricas de aproximación [46].

SparseK. Utiliza una proyección estilo top-k diferenciable sobre puntuaciones de importancia antes de la atención, por lo que solo los pares KV seleccionados participan en la ruta de producto punto costosa [21].

NSA (Atención Dispersa Nativa). Implementa un diseño disperso de tres ramas: rama comprimida, rama seleccionada y rama de ventana local, luego las combina con compuertas aprendidas [44].

SpargeAttn. Filtrado de bloques sin entrenamiento en dos etapas: primero predice interacciones de bloques despreciables, luego aplica poda consciente de softmax para eliminar bloques de baja contribución [48].

FASA. Selección consciente de frecuencia basada en componentes de frecuencia dominantes RoPE, luego aplica atención completa solo en tokens seleccionados [38].

2.1.4. Mecanismos de Memoria con Compuerta

Los mecanismos de memoria con compuerta introducen control dependiente de datos sobre cómo se retiene, olvida y actualiza la información en arquitecturas lineales o recurrentes.

GLA (Atención Lineal con Compuerta). GLA aumenta la atención lineal con una compuerta de decaimiento multiplicativa dependiente de datos. Esto aborda directamente la sobrecarga de memoria en estados recurrentes aditivos, permitiendo que el modelo controle activamente qué información retener a través del tiempo [41].

DeltaNet. DeltaNet reemplaza la acumulación pura con una regla delta: la memoria se corrige comparando el valor recuperado contra el valor deseado. Esto puede interpretarse como corrección de errores en línea y mejora el recuerdo asociativo [42].

Gated DeltaNet. Gated DeltaNet combina ambas ideas: una compuerta de decaimiento controla el olvido mientras que la regla delta controla la escritura dirigida. Esto se presenta como el diseño recurrente lineal más equilibrado porque soporta tanto el borrado rápido como las actualizaciones precisas simultáneamente [43].

HGRN2. HGRN2 utiliza compuertas de olvido jerárquicas acotadas inferiormente con expansión de estado de producto externo. Los límites jerárquicos fomentan que diferentes capas se especialicen en diferentes escalas de tiempo [25].

FoX (Transformador de Olvido). FoX mantiene atención softmax completa pero añade una compuerta de olvido en el espacio de logit. Esto inyecta dinámica consciente de recencia en la canalización estándar sin perder la expresividad de atención completa, aunque mantiene la complejidad cuadrática [17].

Atención con Compuerta después de SDPA. La arquitectura final aplica una compuerta sigmoide después de la atención producto-escala escalada. Los beneficios incluyen la no linealidad adicional que rompe parte del cuello de botella de rango bajo en la ruta de salida, y la compuerta ayuda a suprimir el fenómeno de sumidero de atención [26].

2.2. Algoritmos de Optimización Avanzados

El paisaje de optimización moderna en aprendizaje profundo se ha expandido más allá de SGD y AdamW, con varios desarrollos que abordan desafíos específicos.

Reducción de Varianza. Adan reformula la aceleración Nesterov para aprendizaje profundo, ADOPT corrige problemas teóricos de no convergencia de Adam, y AdEMAMix introduce mezcla de historias duales EMA [39, 34, 24].

Variantes Eficientes en Memoria. Adafactor y GaLore reducen el estado del optimizador mediante factorización o proyección de rango bajo, mientras que Lion utiliza actualizaciones basadas en signos [30, 49, 6].

Enfoques Sin Programador. Schedule-Free AdamW elimina la necesidad de diseño de programador explícito integrando la dinámica de programación en las actualizaciones [8].

Métodos Conscientes de la Curvatura. Sophia, Shampoo y SOAP incorporan información de segundo orden aproximada mediante estimaciones de Hessiano diagonal, estadísticas estructuradas de Kronecker o seguimiento de base propia [12, 36, 18].

Optimizadores Orientados a la Geometría. Muon y Turbo-Muon remodelan explícitamente la geometría de actualización mediante ortogonalización, útil cuando la estructura matricial importa para dar forma a representaciones [31, 4].

3. Arquitectura Centrada en Configuración

3.1. Contrato de Esquema y Validación

El sistema se basa en un contrato de esquema YAML estricto que impone experimentos válidos fallando rápido en configuraciones inválidas. El esquema está estructurado jerárquicamente con `additionalProperties: false` en todos los niveles para eliminar el comportamiento silencioso de "tragar" parámetros desconocidos.

Los componentes principales del esquema son:

- **model**: Arquitectura del codificador (tamaño oculto, capas, tipo de normalización, tipo de mezclador, parámetros específicos del mezclador).
- **tokenizer**: Tokenización y vocabulario (tipo, ruta del modelo, ruta del vocabulario, longitud máxima).
- **training**: Tiempo de ejecución de entrenamiento (tamaño de lote, acumulación, programador, cortado, repetición, puntos de control).
- **optimizer**: Configuración del optimizador (clase, hiperparámetros con prefijo).
- **data**: Carga de datos (directorio de caché, tamaño máximo de secuencia, probabilidad MLM).
- **deploy**: Opciones de despliegue (cuantización, ruta del modelo).
- **sbert**: Configuración específica de SBERT (modo, ruta del corpus, hiperparámetros).

3.2. Validación de Esquema

La validación ocurre en la carga de configuración utilizando el validador PyYAML. Las reglas principales incluyen:

- **Validación de tipo**: Todos los campos deben coincidir con los tipos declarados (entero, flotante, booleano, cadena, enumeración).
- **Validación de rango**: Los campos numéricos deben respetar las restricciones mínimas/máximas (por ejemplo, `batch_size` ≥ 1, `learning_rate` > 0).
- **Validación de enumeración**: Los campos de enumeración deben elegir de conjuntos de opciones predefinidos (por ejemplo, `norm_type` $\in$ {`layer_norm`, `dynamic_tanh`, `derf`}).
- **Validación de estructura**: Los objetos anidados deben cumplir con sus esquemas anidados.
- **Validación de propiedades adicionales**: Se rechazan campos no declarados explícitamente.

4. Taxonomía de Arquitectura e Implementación

4.1. Familias de Atención y Mezclador de Secuencias

Este sistema implementa diecisiete arquitecturas distintas de mezclador de secuencias organizadas en cinco categorías funcionales que reflejan tendencias de investigación en el diseño de modelado de secuencias. La organización taxonómica refleja la comprensión evolutiva de cómo equilibrar expresividad, eficiencia computacional y restricciones de memoria.

1. **Líneas Base de Atención Densa**: Atención softmax estándar y atención sigmoide proporcionan contextualización global completa a costo computacional cuadrático, sirviendo como líneas base de referencia para comparación con alternativas más eficientes.

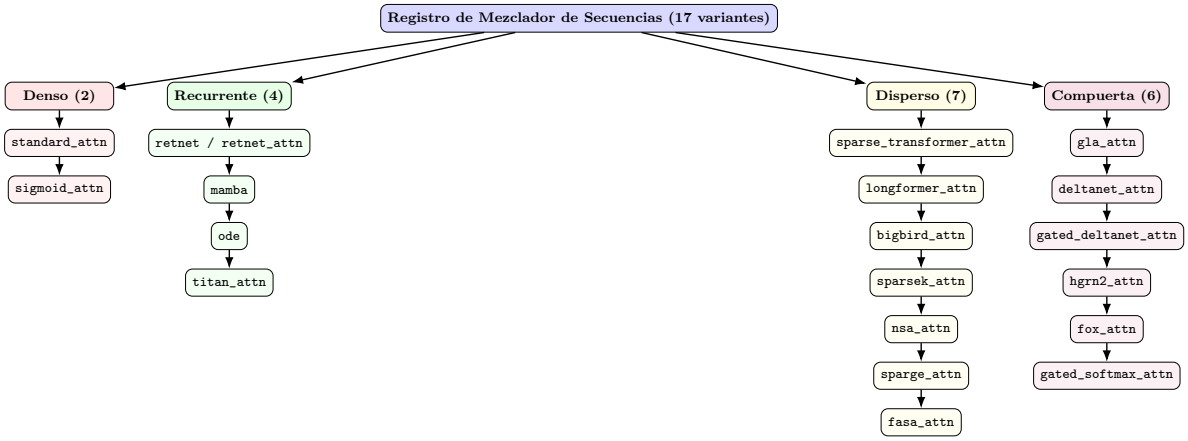

Figura 1: Taxonomía integral de diecisiete arquitecturas de mezclador de secuencias soportadas. Las líneas base densas proporcionan enrutamiento global completo a costo cuadrático. Las arquitecturas recurrentes permiten inferencia de tiempo constante mediante compresión de estado. Las variantes dispersas reducen complejidad mediante patrones estructurados o selección de tokens. Los mecanismos con compuerta introducen control dependiente de datos sobre la retención y olvido de memoria.

2. **Arquitecturas Recurrentes y Retentivas**: RetNet, Mamba, bloques estilo ODE y Titans mantienen representaciones de estado que permiten costo de inferencia $\mathcal{O}(1)$ mientras preservan expresividad a través de dinámica recurrente, parámetros selectivos o adaptación de memoria en tiempo de prueba.

3. **Patrones de Atención Dispersa**: Siete variantes dispersas (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) reducen la complejidad cuadrática mediante dispersión estructurada, selección de tokens o estrategias de poda sin entrenamiento.

4. **Mecanismos de Memoria con Compuerta**: Seis arquitecturas con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax) introducen control dependiente de datos sobre la retención de memoria, el olvido y la fuerza de actualización.

4.2. Atención Estándar

Dadas matrices proyectadas (Q, K, V):

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Este es el mecanismo de línea base para el enrutamiento de contenido [35].

4.3. Atención Sigmoide

La atención sigmoide elimina la normalización de probabilidad por filas:

$$\mathrm{SigmoidAttn}(Q, K, V) = \sigma\left(\frac{QK^\top}{\sqrt{d_k}} + b\right) V$$

y tiene diferentes requisitos de estabilidad de entrenamiento, a menudo con normalización adicional [27].

4.4. Formulación Retentiva

RetNet utiliza retención con matriz de decaimiento D:

$$\text{Retention}(Q, K, V) = \left(QK^\top \odot D\right) V$$

con forma recurrente:

$$S_n = \gamma S_{n-1} + k_n^\top v_n, \qquad o_n = q_n S_n$$

permitiendo inferencia recurrente de bajo costo [33, 40].

4.5. SSM Selectivo (Mamba)

La recurrencia de espacio de estado selectivo discreto es:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t$$

donde $(\bar{A}_t, \bar{B}_t, C_t)$ dependen de la entrada, preservando el escalado de tiempo lineal con escaneo consciente del hardware [10, 13].

4.6. Actualizaciones Continuas Estilo ODE

Enmarcación de profundidad continua:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t)$$

con integradores RK prácticos para ejecución discreta [47].

4.7. Memoria en Tiempo de Prueba (Titans)

Una actualización aumentada con memoria puede escribirse:

$$M_t = (1 - \alpha_t) M_{t-1} + S_t, \qquad S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t)$$

para adaptar la memoria en tiempo de inferencia [2, 1].

4.8. Extensiones Dispersas y con Compuerta en la Base de Código Actual

El registro de mezclador ahora incluye bloques dispersos: `sparse_transformer_attn`, `longformer_attn`, `bigbird_attn`, `sparsek_attn`, `nsa_attn`, `sparge_attn`, y `fasa_attn`; y bloques con compuerta: `gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `hgrn2_attn`, `fox_attn`, y `gated_softmax_attn`.

La implementación impone una política de ejecución explícita para métodos dispersos sin entrenamiento: `fasa_attn` y `sparge_attn` son solo de evaluación/inferencia y lanzan errores de tiempo de ejecución si se usan mientras el modelo está en modo de entrenamiento.

5. Familias de Optimizadores y Dinámica de Entrenamiento

La optimización de arquitecturas de transformador altamente parametrizadas presenta desafíos significativos debido a paisajes de pérdida no convexos, puntos de silla y heterogeneidad de bloques a través de grupos de parámetros. El espectro de Hessiano varía dramáticamente entre incrustaciones, pesos de atención, capas de normalización y redes de retroalimentación, creando caminos de optimización donde ciertas dimensiones son exponencialmente más agudas o planas que otras. Este sistema aborda estos desafíos a través de un marco unificado que admite veinte familias de optimizadores que abarcan múltiples trayectorias algorítmicas.

5.1. Organización Taxonómica de Optimizadores Soportados

El soporte de optimizadores está organizado en seis categorías algorítmicas que reflejan direcciones de investigación modernas en optimización de aprendizaje profundo:

1. **Líneas Base Clásicas**: SGD con momentum, Adam y AdamW definen el suelo de comparación contra el cual se evalúan métodos más nuevos. Estos métodos son bien entendidos pero exhiben limitaciones incluyendo sensibilidad a programadores de tasa de aprendizaje, robustez pobre a curvatura heterogénea (SGD) y alta sobrecarga de memoria para modelos grandes (familia Adam).

2. **Momentum Avanzado y Reducción de Varianza**: Adan, ADOPT, AdEMAMix, MARS y Cautious AdamW (2024–2025) abordan limitaciones específicas de métodos clásicos. Adan reformula la aceleración Nesterov, ADOPT corrige la no convergencia teórica de Adam, AdEMAMix introduce mezcla de historias duales EMA, MARS trae reducción de varianza al entrenamiento a gran escala, y variantes Cautious enmascaran actualizaciones a direcciones consistentes en signo.

3. **Optimizadores Eficientes en Memoria**: Adafactor, GaLore y Lion reducen el estado del optimizador mediante factorización, proyección de rango bajo o actualizaciones basadas en signos. Estos métodos son críticos cuando VRAM está dominado por estado del optimizador más que activaciones.

4. **Sin Programador y Sin Parámetros**: Schedule-Free AdamW y Prodigy eliminan el diseño de programador explícito o ajuste de tasa de aprendizaje haciendo que la dinámica de actualización absorba estas responsabilidades, reduciendo la complejidad de búsqueda de hiperparámetros.

5. **Conscientes de la Curvatura y de Segundo Orden**: Sophia, Shampoo y SOAP incorporan información de segundo orden aproximada mediante estimaciones de Hessiano diagonal, estadísticas estructuradas de Kronecker o seguimiento de base propia, mejorando la condición a costo de complejidad de implementación.

6. **Orientados a Geometría**: Muon y Turbo-Muon remodelan explícitamente la geometría de actualización mediante ortogonalización, útil cuando la estructura matricial importa para dar forma a representaciones.

5.2. Formulación Adaptativa Central

Muchos optimizadores soportados comparten formulación fundamental de seguimiento de momentos. Los optimizadores de familia Adam rastrean primer y segundo momentos de gradientes:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

seguidos por actualizaciones precondicionadas con corrección de sesgo:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

donde η_t es el programador de tasa de aprendizaje, ϵ es constante de estabilidad numérica, y λ es coeficiente de decaimiento de pesos [20].

6. Cuantización y Despliegue

La pila de despliegue utiliza empaquetado de pesos ternarios más cuantización de activaciones INT8 para artefactos eficientes.

6.1. Cuantización Ternaria

Dado tensor de pesos W, un escalado práctico es:

$$s = \text{mean}(|W|), \qquad \tilde{W} = \text{clip}\left(\text{round}\left(\frac{W}{s}\right), -1, 1\right)$$

que aproxima actualizaciones de bajo bits estilo BitNet [37]. La mapeo empaquetada utiliza dos bits por símbolo de peso para eficiencia de almacenamiento.

6.2. Cuantización de Activaciones

Para activaciones x:

$$q = \text{round}\left(x \cdot \frac{127}{\text{máx}(|x|) + \epsilon}\right), \qquad q \in [-128, 127]$$

con decuantización $x \approx q/\alpha$.

6.3. Estimaciones de Tamaño

Para N parámetros:

$$\text{Tamaño FP32} \approx 4N, \quad \text{Tamaño FP16} \approx 2N, \quad \text{Tamaño 1.58-bit} \approx \frac{1{,}58}{8}N$$

antes de metadatos y sobrecarga de empaquetado. Esto se alinea con objetivos de despliegue ligero [29, 5].

7. Tareas Posteriores SBERT

La incrustación de oraciones se basa en entrenamiento estilo siamés [28]. Para par de oraciones (s_1, s_2) con incrustaciones (e_1, e_2):

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}$$

y pérdida de coseno estilo regresión:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

con $y \in [-1, 1]$ en esta canalización.

Modos posteriores soportados:

- **Similitud**: puntuación por pares entre dos oraciones.
- **Búsqueda**: k vecinos más cercanos sobre un corpus.
- **Agrupamiento**: agrupación de incrustaciones (por ejemplo, k-means).
- **Codificación**: exportación de incrustaciones persistentes para recuperación posterior.

8. Tablas Resumen

8.1. Resumen de Atención y Mezclador de Secuencias

Tipo	Ecuación Central	Entrenamiento	Inferencia	Notas
Atención Estándar	$\mathrm{softmax}(QK^\top/\sqrt{d_k})V$	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$/paso	Enrutamiento global expresivo de línea base [35].
Atención Sigmoide	$\sigma(QK^\top/\sqrt{d_k}+b)V$	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$/paso	Compuerta elemento por elemento; a menudo necesita norma de estabilización [27].
RetNet	$(QK^\top \odot D)V$	$\mathcal{O}(n^2 d)$ o por bloques	$\mathcal{O}(1)$/paso	Forma dual paralela/recurrente con retención de decaimiento [33].
Mamba	$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$	$\mathcal{O}(nd)$	$\mathcal{O}(1)$/paso	Espacio de estado selectivo con escaneo consciente del hardware [10].
Bloque estilo ODE	$\frac{dh}{dt} = f_\theta(h,t)$	depende del solucionador	depende del solucionador	Interpretación de profundidad continua; integración RK [47].
Memoria Titans	$M_t = (1-\alpha_t)M_{t-1} + S_t$	aprox. $\mathcal{O}(nd)$	centrada en recuperación	Actualizaciones de memoria en tiempo de prueba con dinámica impulsada por sorpresa [2].

8.2. Resumen de Optimizadores

Optimizador	Familia	Costo de Estado	Idea Clave	Ref
AdamW	Adaptativo momento primero/segundo	Alto	Línea base con decaimiento de pesos desacoplado	[20]
RAdam	Adaptativo corregido en varianza	Alto	Rectifica varianza adaptativa temprana	[19]
Adan	Momentum + reducción de varianza	Alto	Actualización adaptativa estilo Nesterov	[39]
ADOPT	Variante de Adam	Alto	Actualizaciones reordenadas con garantías de convergencia mejoradas	[34]
AdEMAMix	Adaptativo multi-EMA	Alto	Mezcla EMAs de horizonte corto y largo	[24]
MARS	Precondicionado reducido en varianza	Alto	Corrección de momentum recursiva	[45]
Cautious AdamW	Momentum enmascarado	Alto	Aplica actualizaciones solo en direcciones consistentes en signo	[16]
Schedule-free AdamW	Adaptativo sin programador	Alto	Elimina dependencia de programador LR explícito	[8]
Adafactor	Adaptativo eficiente en memoria	Medio	Segundos momentos factorizados para tensores de matriz	[30]
GaLore AdamW	Proyección de gradiente de rango bajo	Medio	Optimiza en espacio de gradiente proyectado de rango bajo	[49]
Prodigy	Adaptación sin parámetros	Medio	Calibración de paso adaptativa a distancia	[22]

Optimizador	Familia	Costo de Estado	Idea Clave	Ref
Lion	Momentum de signo	Bajo	Actualización de momentum de signo, estado reducido	[6]
Sophia	Segundo orden aprox.	Medio	Precondicionamiento de Hessiano diagonal con recorte	[18]
Shampoo	Precondicionador de matriz	Alto	Estadísticas de segundo orden estructuradas de Kronecker	[12]
SOAP	Shampoo + base Adam	Alto	Seguimiento estilo Adam en base de precondicionador	[36]
Muon	Basado en ortogonalidad	Medio	Actualizaciones matriciales ortogonalizadas	[31]
Turbo-Muon	Ortogonalización acelerada	Medio	Aceleración de Newton-Schulz precondicionada	[4]

9. Discusión

Las elecciones de diseño en Transformer Encoder Frankenstein reflejan varias tensiones de ingeniería e investigación en herramientas modernas de aprendizaje profundo.

9.1. Compromisos de Diseño Impulsado por Esquemas

El enfoque basado primero en esquemas proporciona beneficios significativos de reproducibilidad imponiendo contratos explícitos y fallando rápido en configuraciones inválidas. Sin embargo, este enfoque también introduce rigidez: agregar nuevas arquitecturas u optimizadores requiere extensiones de esquema más que argumentos de línea de comandos sueltos. El sistema de hiperparámetros con prefijo permite control granular pero aumenta la complejidad de configuración para usuarios acostumbrados a interfaces más simples.

La decisión de imponer `additionalProperties: false` en todos los niveles de esquema elimina el comportamiento silencioso de "tragar" parámetros que ha plagado sistemas de configuración anteriores, pero esta estrictitud requiere mantenimiento cuidadoso del esquema al extender capacidades del sistema. Cada nuevo mecanismo de atención o variante de optimizador debe integrarse adecuadamente en el marco de validación, incluyendo definiciones de campos de esquema con tipos y restricciones apropiados, mapeo de hiperparámetros con prefijo para grupos específicos de optimizador, valores predeterminados alineados con mejores prácticas de investigación, y cadenas de documentación para renderizado de interfaz web.

9.2. Cobertura Arquitectónica y Brechas

Las diecisiete arquitecturas de mezclador implementadas abarcan direcciones principales de investigación en modelado de secuencias, pero ciertas brechas permanecen. El sistema carece de arquitecturas híbridas recientes como Griffin [32] y Jamba [11], que combinan compuertas con modelos de espacio de estado. El enrutamiento MoE (Mezcla de Expertos) está implementado para capas FFN pero no para cómputo de atención, donde el trabajo reciente ha mostrado beneficios [15].

La cobertura de atención dispersa es integral, pero la implementación de métodos sin entrenamiento (FASA, SpargeAttn) lanza errores de tiempo de ejecución durante el entrenamiento, reflejando restricciones arquitectónicas: estos métodos requieren puntos de control preentrenados de modelos de atención completa o procedimientos de ajuste específicos que no están automatizados actualmente.

La cobertura de mecanismo con compuerta es fuerte a través de categorías principales (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).

9.3. Fragmentación del Paisaje de Optimizadores

El soporte para veinte familias de optimizadores a través de seis categorías algorítmicas demuestra integralidad pero también destaca el estado fragmentado de investigación de optimización. Los usuarios enfrentan una complejidad de decisión significativa al elegir entre métodos de reducción de varianza (Adan, MARS), variantes eficientes en memoria (GaLore, Adafactor) y enfoques conscientes de la curvatura (Sophia, Shampoo). El sistema de hiperparámetros con prefijo, aunque poderoso, requiere comprensión de qué parámetros son relevantes para cada clase de optimizador.

La calidad de implementación varía a través de optimizadores: métodos clásicos (AdamW, SGD con momentum) están altamente optimizados en PyTorch, mientras que métodos más nuevos (Muon, Turbo-Muon, SOAP) pueden requerir implementaciones personalizadas que afectan estabilidad numérica y características de rendimiento.

9.4. Consideraciones de Despliegue y Producción

La canalización de cuantización demuestra preocupaciones prácticas de despliegue pero hace compromisos de ingeniería específicos. El empaquetado de pesos ternarios reduce el almacenamiento a aproximadamente 1,58 bits por parámetro, pero esta compresión agresiva puede degradar el rendimiento, especialmente para modelos más pequeños donde el error de cuantización es más significativo. La implementación actual aplica cuantización uniformemente a través de tipos de parámetros.

Los flujos de trabajo SBERT proporcionan utilidad práctica para tareas de similitud semántica y recuperación, pero la implementación asume estrategias de agrupamiento estándar (token CLS, agrupamiento promedio). Avances recientes como incrustaciones Matryoshka [23] y refinamientos de aprendizaje contrastivo [14] no están incorporados todavía.

9.5. Desafíos de Integración y Extensibilidad

La estructura de base de código actual, aunque funcional, presenta desafíos de mantenimiento a medida que se expanden familias de arquitectura y optimizador. El patrón de despachador para selección de mezclador y enrutamiento de optimizador maneja extensibilidad pero arriesga convertirse en un "fregadero de cocina" de lógica condicional. Versiones futuras se beneficiarían de arquitecturas basadas en complementos donde nuevos mezcladores y optimizadores podrían registrarse declarativamente más que modificar lógica de despacho central.

La interfaz de configuración web proporciona mejoras significativas de usabilidad pero introduce complejidad de despliegue: ejecutar Streamlit junto con trabajos de entrenamiento requiere recursos adicionales e consideraciones de infraestructura que pueden no ser apropiadas para todos los entornos, particularmente clústeres HPC sin acceso web.

10. Conclusión

Transformer Encoder Frankenstein presenta una plataforma de experimentación unificada basada en configuración que aborda desafíos críticos en investigación moderna de aprendizaje profundo: fragmentación arquitectónica a través de atención densa, modelos recurrentes, patrones dispersos y mecanismos con compuerta; complejidad del paisaje de optimizadores que abarca líneas base clásicas, métodos de reducción de varianza, variantes eficientes en memoria, enfoques sin programador, algoritmos conscientes de la curvatura y métodos orientados a geometría; y flujos de trabajo de extremo a extremo que abarcan cuantización y aplicaciones de incrustación de oraciones.

Las contribuciones principales del sistema son:

1. **Diseño Basado Primero en Esquemas**: Un contrato de configuración basado en YAML estricto con validación y enrutamiento de hiperparámetros con prefijo que permite experimentos reproducibles a través de diecisiete arquitecturas de mezclador y veinte familias de optimizadores.

2. **Soporte de Arquitectura Integral**: Implementación que abarca categorías principales de investigación incluyendo líneas base densas (atención estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, bloques estilo ODE, memoria aumentada con memoria), patrones dispersos (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).

3. **Marco de Enrutamiento de Optimizadores**: Sistema de grupos de hiperparámetros con prefijo que permite control por grupo de parámetros a través de veinte optimizadores que abarcan líneas base clásicas, reducción de varianza, eficiencia de memoria, programación libre, segundo orden y métodos orientados a geometría.

4. **Cuantización y Despliegue**: Canalización integrada que soporta empaquetado de pesos ternarios y cuantización de activaciones INT8 con estimaciones de tamaño para despliegue ligero.

5. **Flujos de Trabajo de Incrustación de Oraciones**: Canalizaciones de entrenamiento e inferencia inspiradas en SBERT que soportan similitud, recuperación, agrupamiento y exportación de incrustaciones.

6. **Interfaz de Configuración Interactiva**: Servidor web basado en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y generación de comandos CLI.

Este trabajo demuestra que un diseño basado en esquemas riguroso puede unificar experimentación a través de arquitecturas y optimizadores diversos mientras mantiene reproducibilidad. El sistema proporciona una plataforma sólida para exploración sistemática de diseño de transformadores, permitiendo a investigadores y profesionales iterar rápidamente en configuraciones experimentales sin comprometer condiciones reproducibles.

El desarrollo futuro se centrará en extender la cobertura arquitectónica (agregando variantes híbridas y mecanismos MoE para atención), mejorar la canalización de despliegue (soportando estrategias de cuantización más sofisticadas y métodos de inferencia eficientes), y refinar la interfaz de configuración (añadiendo asistentes guiados para selección de arquitectura/optimizador y exportación de configuraciones compartibles).

Bibliografía

[1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, . URL http://arxiv.org/abs/2501.00663.

[2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, . URL https://arxiv.org/abs/2501.00663. Version Number: 1.

[3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

[4] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbomuon: Accelerating orthogonality-based optimization with pre-conditioning. URL https://arxiv.org/abs/2512.04632. Version Number: 1.

[5] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory. URL http://arxiv.org/abs/2502.10001.

[6] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms. URL https://arxiv.org/abs/2302.06675. Version Number: 4.

[7] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL https://arxiv.org/abs/1904.10509.

[8] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. URL https://arxiv.org/abs/2405.15682. Version Number: 4.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. URL http://arxiv.org/abs/1810.04805.

[10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. URL https://arxiv.org/abs/2312.00752. Version Number: 2.

[11] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL https://arxiv.org/abs/2403.19887.

[12] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. URL https://arxiv.org/abs/1802.09568. Version Number: 2.

[13] Sukjun Hwang, Aakash Lahoti, Tri Dao, and Albert Gu. Hydra: Bidirectional state space models through generalized matrix mixers. URL http://arxiv.org/abs/2407.09941.

[14] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL https://arxiv.org/abs/2004.11362.

[15] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL https://arxiv.org/abs/2103.16716.

[16] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. URL https://arxiv.org/abs/2411.16085. Version Number: 4.

[17] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL https://arxiv.org/abs/2503.02130.

[18] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, . URL https://arxiv.org/abs/2305.14342. Version Number: 4.

[19] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, . URL https://arxiv.org/abs/1908.03265. Version Number: 4.

[20] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. URL https://arxiv.org/abs/1711.05101. Version Number: 3.

[21] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL https://arxiv.org/abs/2406.16747.

[22] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner. URL https://arxiv.org/abs/2306.06101. Version Number: 4.

[23] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL https://arxiv.org/abs/2205.13147.

[24] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older. URL https://arxiv.org/abs/2409.03137. Version Number: 2.

[25] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL https://arxiv.org/abs/2404.07904.

[26] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL https://arxiv.org/abs/2505.06708.

[27] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention. URL https://arxiv.org/abs/2409.04431. Version Number: 2.

[28] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. URL http://arxiv.org/abs/1908.10084.

[29] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications. URL http://arxiv.org/abs/2601.03290.

[30] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. URL https://arxiv.org/abs/1804.04235. Version Number: 1.

[31] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon. URL https://arxiv.org/abs/2505.23737. Version Number: 1.

[32] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL https://arxiv.org/abs/2402.19427.

[33] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. URL https://arxiv.org/abs/2307.08621. Version Number: 4.

[34] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any $_2$ with the optimal rate. URL https://arxiv.org/abs/2411.02853. Version Number: 3.

[35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. URL https://arxiv.org/abs/1706.03762. Version Number: 7.

[36] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam. URL https://arxiv.org/abs/2409.11321. Version Number: 2.

[37] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. URL http://arxiv.org/abs/2310.11453.

[38] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL https://arxiv.org/abs/2602.03152.

[39] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. URL https://arxiv.org/abs/2208.06677. Version Number: 5.

[40] Haiqi Yang, Zhiyuan Li, Yi Chang, and Yuan Wu. A survey of retentive network. URL http://arxiv.org/abs/2506.06708.

[41] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL https://arxiv.org/abs/2312.06635.

[42] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL https://arxiv.org/abs/2406.06484.

[43] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL https://arxiv.org/abs/2412.06464.

[44] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL https://arxiv.org/abs/2502.11089.

[45] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models. URL https://arxiv.org/abs/2411.10438. Version Number: 4.

[46] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL https://arxiv.org/abs/2007.14062.

[47] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. 35(16):14393–14401. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL https://ojs.aaai.org/index.php/AAAI/article/view/17692.

[48] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL https://arxiv.org/abs/2502.18137.

[49] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. URL https://arxiv.org/abs/2403.03507. Version Number: 2.